

Task 1: Cài đặt gensim

```
%pip install gensim
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.0.1)
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
27.9/27.9 MB 22.2 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

Thử nghiệm module downloader của gensim:

```
import gensim.downloader as api
```

Task 2: Thiết lập WordEmbedder theo hướng dẫn

```
from statistics import mean

class WordEmbedder:
    def __init__(self, model_name: str):
        self.model = api.load(model_name)

    #Hàm trả về vector nhúng của từ
    def get_vector(self, word: str):
        if word in self.model:
            return self.model[word]
        else:
            return f"{word} is not in vocab!"

    #Hàm trả về độ tương đồng giữa 2 từ
    def get_similarity(self, word1: str, word2: str):
        if word1 in self.model and word2 in self.model:
            return self.model.similarity(word1, word2)

    #Hàm gọi lấy top_n(mặc định 10) từ gần nhất với word.
    def get_most_similar(self, word: str, top_n: int=10):
        if word in self.model:
            return self.model.most_similar(word, topn=top_n)

    #Hàm tokenize dựa theo SimpleTokenizer ở lab 1
    def tokenize(self, text:str) -> list[str]:
        return text.lower().split()

    #Hàm nhúng document và trả về danh sách mean theo element-wise của các vector từ
    def embed_document(self, document: str):
        tokens = self.tokenize(document)
        word_vectors = []
        for token in tokens:
            if token in self.model:
                word_vectors.append(self.model[token])

        #Kiểm tra document có chứa từ nào trong vocab không.
        if not word_vectors:
            return [0] * self.model.vector_size

        element_wise_mean_py = list(map(mean, zip(*word_vectors)))

        return element_wise_mean_py
```

Triển khai WordEmbedder:

```
wordEmbedder = WordEmbedder("glove-wiki-gigaword-50")

print(f"get word vector of king: {wordEmbedder.get_vector('king')}")
print(f"get similarity of king and queen: {wordEmbedder.get_similarity('king', 'queen')}")
print(f"get similarity of king and man: {wordEmbedder.get_similarity('king', 'man')}")
print(f"get top 10 most similar to computer: {wordEmbedder.get_most_similar('computer')}")
print(f"get document vector of The queen rules the country. : {wordEmbedder.embed_document('The queen rules the country.')}")
```

```

get word vector of king: [ 0.50451  0.68607 -0.59517 -0.022801 0.60046 -0.13498 -0.08813
 0.47377 -0.61798 -0.31012 -0.076666 1.493 -0.034189 -0.98173
 0.68229 0.81722 -0.51874 -0.31503 -0.55809 0.66421 0.1961
-0.13495 -0.11476 -0.30344 0.41177 -2.223 -1.0756 -1.0783
-0.34354 0.33505 1.9927 -0.04234 -0.64319 0.71125 0.49159
0.16754 0.34344 -0.25663 -0.8523 0.1661 0.40102 1.1685
-1.0137 -0.21585 -0.15155 0.78321 -0.91241 -1.6106 -0.64426
-0.51042 ]
get similarity of king and queen: 0.7839043140411377
get similarity of king and man: 0.5309377312660217
get top 10 most similar to computer: [('computers', 0.9165045022964478), ('software', 0.8814992904663086), ('technology', 0.
get document vector of The queen rules the country. : [np.float32(0.06438001), np.float32(0.43381), np.float32(-0.779435), r

```

Nhận xét về kết quả:

- "king" gần "queen" hơn gần "man": $0.78 > 0.53$
- Các từ gần "computer" nhất là "software", "technology", "electronic",... đều liên quan đến công nghệ thông tin, máy tính.

=> Các kết quả trên cho thấy mô hình khá tốt trong việc biểu diễn ý nghĩa từ.

Task 3: Huấn luyện mô hình Word2Vec thủ công

```

import os
from gensim.models import Word2Vec
from gensim.utils import simple_preprocess

class SentenceStream:
    def __init__(self, filepath):
        self.filepath = filepath

    def __iter__(self):
        with open(self.filepath, "r", encoding="utf-8") as f:
            for line in f:
                line = line.strip()
                if line:
                    yield simple_preprocess(line)

data_path = "./data/en_ewt-ud-train.txt"
results_dir = "results"
os.makedirs(results_dir, exist_ok=True)

sentences = SentenceStream(data_path)

print("Training Word2Vec model...")
model = Word2Vec(
    sentences=sentences,
    vector_size=100,
    window=5,
    min_count=3,
    workers=8,
    sg=1
)

print("Huấn luyện xong!")

model_path = os.path.join(results_dir, "word2vec_ewt.model")
model.save(model_path)
print("Done saving")

# Demo
print("Tìm các từ gần nghĩa với 'computer'")
if "computer" in model.wv:
    for w, s in model.wv.most_similar("computer", topn=5):
        print(f"{w:15s} {s:.4f}")
else:
    print("Từ 'computer' không có trong từ vựng mô hình.")

print("Ví dụ: king - man + woman = ?")
try:
    result = model.wv.most_similar(positive=["king", "woman"], negative=["man"], topn=1)
    print(f"Kết quả gần đúng: {result[0][0]} ({result[0][1]:.4f})")
except KeyError:
    print("Một trong các từ không tồn tại trong từ vựng mô hình.")

```

```

Training Word2Vec model...
Huấn luyện xong!
Done saving
Tìm các từ gần nghĩa với 'computer'
fill 0.9972

```

```

coffee      0.9967
break       0.9967
crate       0.9962
transfer    0.9962
Ví dụ: king - man + woman ≈ ?
Kết quả gần đúng: graphic (0.9896)

```

Nhận xét:

- Các từ gần nghĩa với "computer" là "fill", "coffee", ... không có quan hệ ngữ nghĩa.
- ví dụ sử dụng king - man + woman có kết quả mong muốn là queen thì ra graphic.

=> Các kết quả cho thấy có thể từ vựng của bộ dữ liệu sử dụng chưa được hoàn thiện và đầy đủ so với mô hình pre-trained, dẫn đến kết quả kém hơn.

Task 4: Scale Word2Vec bằng Spark

```
%pip install spark re
```

```

Collecting spark
  Downloading spark-0.3.2-py3-none-any.whl.metadata (1.3 kB)
ERROR: Could not find a version that satisfies the requirement re (from versions: none)
ERROR: No matching distribution found for re

```

```

import re
from pyspark.sql import SparkSession
from pyspark.ml.feature import Tokenizer, Word2Vec
from pyspark.sql.functions import col, lower, regexp_replace, split
# Initialize Spark Session
# ...
spark = SparkSession.builder.appName("Word2VecExample").getOrCreate()
# Load the dataset
# The C4 dataset is in JSON format, with each line being a JSON object.
df = spark.read.json("./data/c4-train.00000-of-01024-30K.json")
# We are interested in the 'text' field.
# ...
# Preprocessing
# 1. Select the text column and convert to lowercase
df = df.select(lower(col("text")).alias("text"))
# 2. Remove punctuation and special characters
df = df.withColumn("text", regexp_replace(col("text"), "[^a-zA-Z\\s]", ""))
# 3. Split the text into an array of words
df = df.withColumn("words", split(col("text"), "\\s+").cast("array<string>"))
# ...
# Configure and train the Word2Vec model
word2vec = Word2Vec(vectorSize=100, minCount=5, inputCol="words", outputCol="embedding")
# ...
# Demonstrate the model
model = word2vec.fit(df)
# ...
# Find synonyms for a word
synonyms = model.findSynonyms("computer", 5)
synonyms.show()
# ...
# Stop the Spark session
# spark.stop()

```

```

+-----+-----+
|      word|      similarity|
+-----+-----+
|  desktop|0.6940192580223083|
|programming|0.6447052955627441|
|  uwowned|0.6285549998283386|
|  computers|0.6282289624214172|
|  software|0.6238794922828674|
+-----+-----+

```

Nhận xét:

- Mô hình train lâu tới ~7 phút so với pre-train (không cần train) và train thủ công (dữ liệu nhỏ hơn)
- kết quả khá tốt đối với nhiệm vụ tìm từ gần nghĩa cho "computer" với các từ cũng liên quan về công nghệ thông tin như "desktop", "programming", ...

Task 5: Trực quan hóa bằng PCA, vẽ scatter-plot:

```
%pip install numpy pandas scikit-learn matplotlib
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.2.5)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Lấy embedding matrix từ Spark Word2Vec model
vectors = model.getVectors()

pdf = vectors.toPandas()

words = pdf["word"].values
X = np.vstack(pdf["vector"].values)
```

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

print("Explained variance ratio:", pca.explained_variance_ratio_)
```

```
Explained variance ratio: [0.0876027  0.08600685]
```

Nhận xét:

- Trong 2 chiều, các từ trở nên khá tập trung, khó phân cụm

```
plt.figure(figsize=(12, 10))
plt.scatter(X_pca[:, 0], X_pca[:, 1], alpha=0.6)

# annotate một số từ (tránh quá rối)
for i, word in enumerate(words[:100]):
    plt.annotate(word, (X_pca[i, 0], X_pca[i, 1]), fontsize=9)

plt.title("PCA Visualization of Word2Vec Embeddings (Spark)")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.grid(True)
plt.show()
```

